

МИНОБРАЗОВАНИЯ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Математический факультет

Кафедра функционального анализа и операторных уравнений

Программная реализация на языке JavaScript некоторых алгоритмов
генерации ФОС по математике

Бакалаврская работа

Направление 02.03.01 Математика и компьютерные науки

Профиль Математическое и компьютерное моделирование

Зав. кафедрой Ка д. ф.-м. н., проф. М.И. Каменский __. __.20__ г.

Обучающийся Китаева В.Д. Китаева

Руководитель Ка к. ф.-м. н., проф. М.И. Каменский

Воронеж 2026

Содержание

Введение	3
1 Реализация алгоритмов на базе проекта «Час ЕГЭ»	5
2 Вклад автора в расширение каталога	9
2.1 Преимущества программной генерации заданий	9
2.2 Вклад автора в расширение каталога	9
3 Рефакторинг программного кода в среде разработки на JavaScript	35
4 Рефакторинг заданий ОГЭ и ЕГЭ	38
4.1 Вклад автора в рефакторинг на проекте	38
Заключение	56

Введение

Единый государственный экзамен (ЕГЭ) — централизованно проводимый в Российской Федерации экзамен в средних учебных заведениях — школах, лицеях и гимназиях, форма проведения ГИА (Государственной Итоговой Аттестации) по образовательным программам среднего общего образования. Служит одновременно выпускным экзаменом из школы и вступительным экзаменом в вузы.

За два года подготовки к ЕГЭ школьники сталкиваются с дефицитом заданий для подготовки. А учителя со списыванием ответов при решении задач экзамена учениками. Также в список заданий ЕГЭ были добавлены новые текстовые задания, количество которых для прорешивания очень мало. Проект «Час ЕГЭ» [2] позволяет решить все эти проблемы. «Час ЕГЭ» — компьютерный образовательный проект, разрабатываемый при математическом факультете ВГУ в рамках «OpenSource кластера» и предназначенный для помощи учащимся старших классов подготовиться к тестовой части единого государственного экзамена. Задания в «Час ЕГЭ» генерируются случайным образом по специализированным алгоритмам, называемых шаблонами, каждый из которых охватывает множество вариантов соответствующей ему задачи. Для пользователей предназначены четыре оболочки (режима работы): «Случайное задание», «Тесты на печать», «Полный тест» и «Мини-интеграция». «Час ЕГЭ» является полностью открытым (код находится под лицензией GNU GPL 3.0) и бесплатным. В настоящее время в проекте полностью реализованы тесты по математике с кратким ответом (бывшая «часть В»). Планируется с течением времени включить в проект тесты по другим предметам школьной программы.

Традиционный подход к подготовке к ЕГЭ предполагает составление и издание печатных и электронных сборников заданий [13, 14], а также формирование открытых банков задач [3, 5] и онлайн-тренажёров [7]. Однако такой подход имеет существенный недостаток: любой сборник содержит конечное, заранее известное число вариантов, которые быстро становятся известны учащимся и теряют свою

тренировочную ценность. Кроме того, ручное составление качественных задач — трудоёмкий процесс, требующий значительных временных затрат от методистов и преподавателей. В связи с этим в рамках данной работы применяется генеративный подход: вместо создания бесконечных сборников заданий разрабатываются алгоритмы-шаблоны, которые на лету порождают неограниченное число уникальных вариантов задач с заранее известным правильным ответом.

Первая глава этой работы посвящена реализации алгоритмов на базе проекта «Час ЕГЭ».

Вторая глава представляет решение проблемы нехватки задач ЕГЭ: шаблоны задач, написанные автором, и примеры генерации.

Третья и четвёртая главы посвящены рефакторингу кода проекта «Час ЕГЭ» с целью повышения его читаемости, поддерживаемости и расширяемости. В них рассматриваются применяемые подходы к улучшению существующего кода, а также особенности разработки веб-приложений на JavaScript, что позволяет подготовить проект к дальнейшему развитию и добавлению новых типов задач.

1. Реализация алгоритмов на базе проекта «Час ЕГЭ»

В этой главе мы приводим вспомогательные функции и алгоритм написания шаблона текстовой задачи.

Функции, используемые в проекте

За 10 лет работы над проектом «Час ЕГЭ» была разработана нестандартная библиотека для упрощения многих задач. Далее представлены наиболее используемые функции из неё. Вспомогательные функции `function sluchch(n,k,s)` Возвращает случайное число от `n` до `k` с шагом `s` (по умолчанию 1). Эта функция используется настолько часто, что для неё была придумана сокращённая форма `sl()`. `function slKrome(a,p1,p2,p3)` Возвращает случайное число, кроме `a`. Если `a` – массив, то не содержащееся в нём; если число или строка, то не равное ему; Если функция, принимающая параметр - то не удовлетворяющее ей.

Работа с числами

1. `Number.prototype.chislitlx=function(p1,p2)` Возвращает строку, состоящую из данного числа и подходящего падежа слова `p1`, при этом полученное словосочетанию стоит в падеже `p2` (если не указан - именительный).
2. `Number.prototype.pow=function(n)` Возвращает число в степени `n`.
3. `Number.prototype.sqrt=function(n)` Возвращает квадратный корень из числа.
4. `Number.prototype.sqr=function()` Возвращает квадрат числа.
5. `Number.prototype.abs=function()` Возвращает модуль числа.
6. `Number.prototype.floor=function()` Возвращает число, округленное до целого в меньшую сторону.
7. `Number.prototype.ceil=function()` Возвращает число, округленное до целого в большую сторону.

8. `Number.prototype.pm=function()` Случайным образом возвращает число или ему противоположное.

Работа со строками

1. `Number.prototype.toZagl=function()` Возвращает исходную строку с первой заглавной буквой.
2. `setEquationTask`-отвечает за составление задач с уравнениями.
3. `roots[]` задаёт корни уравнения

Для примера возьмём задание №27482.

Задача №27482. Причалы K и B расположены на водохранилище, расстояние между ними равно 255 км. Моторная лодка отправилась с постоянной скоростью из K в B . Через неделю после прибытия она отправилась тем же путём обратно со скоростью на 4 км/ч больше прежней, сделав по пути остановку на 1 час. В результате она затратила на обратный путь столько же времени, сколько на путь из K в B . Определите скорость моторной лодки на пути из K в B . Ответ дайте в км/ч.

1. Выбираем задание из Открытого Банка Заданий ЕГЭ и копируем его текст.
2. Добавляем ответ в поле `answers` (по умолчанию 0).
3. Инициализируем всех необходимые переменные для задачи (вес, проценты и так далее). Присваиваем им значения при помощи функции `sluchch()` или `slKrome()` (см. главу 2). Для хранения ответа создаём отдельную переменную.
4. Заменяем все числа в тексте на переменные (при помощи `+'.'+`).
5. Обособляем слова, которые не влияют на условия задачи. Это могут быть имена, профессии, транспорт и т.п.

6. Создаём переменные, которые будут отвечать за выбранные в прошлом пункте слова, и заменяем слова на переменные в тексте задачи. Выбираем их значения из массивов при помощи `iz()`
7. Иногда в задании выбранные слова используются в разных падежах. Для этого в проекте существует лексический модуль. Используем на склоняемых словах функцию `sklonlxkand()`. Теперь необходимо указать падеж слов в задании. Также при необходимости заглавной буквы в слове используем `toZagl()`. Если в тексте задачи присутствуют слова, зависящие от числительных, к ним применяется функция `chislitlx()`.
8. Далее составляем общее решение с учетом всех переменных и особенностей задачи.
9. Заготовка шаблона имеет следующий вид.

```
1 (function() { 'use strict'; retryWhileError(function() {  
2   let s=slKrome(10, 300, 1);  
3   let a=sl(1, 20);  
4   let b=sl(1, 7);  
5   let v=Math.sqrt(b**2*a**2+4*b*s*a);  
6     let x=(-b*a+v)/(2*b);  
7   genAssert(x>0, 'Скорость не может быть отрицательной');  
8   genAssertZ1000(x, 'Скорость не может быть слишком дробной');  
9     let the_berthForFloatingVehicle =  
       sklonlxkand(decor.berthForFloatingVehicle.iz());  
10  let the_waterbodyWithoutCurrent =  
       sklonlxkand(decor.waterbodyWithoutCurrent.iz());  
11  let the_activeFloatingVehicle =  
       sklonlxkand(decor.activeFloatingVehicleF.iz());  
12  let the_afterAWhile = decor.afterAWhile.iz();  
13  let the_orderToFind = decor.orderToFind.iz();  
14    NATask.setTask({  
15    text:
```

```

16      '' + the_berthForFloatingVehicle.im.toZagl() +' А и В
расположены на ' + the_waterbodyWithoutCurrent.pe +', '+'
17      'расстояние между ними равно ' + s + ' км. ' +
the_activeFloatingVehicle.ie.toZagl() +' отправилась с постоянной
скоростью из А в В. ' +
18      the_afterAWhile.toZagl() +' после прибытия она отправилась тем же
путём обратно со скоростью на ' + a + ' кмч/ больше прежней, '+'
19      'сделав по пути остановку на ' + chislitlx(b, 'час') + '. В
результате она затратила на обратный путь столько же времени, '+'
20      'сколько на путь из А в В. ' + the_orderToFind.toZagl() +'
скорость ' + the_activeFloatingVehicle.re +' на пути из А в В. Ответ
дайте в кмч/.',
21      answers: x,
22  });
23  NATask.modifiers.allDecimalsToStandard(/*true*/);
24  NATask.modifiers.variativeABC();
25 }, 2000);})();

```

Примеры генерации задний приведены во второй главе.

2. Вклад автора в расширение каталога

В этой главе решается проблема нехватки заданий для подготовки к ЕГЭ, а также приводятся аргументы в пользу программного написания шаблонов для подготовки к ЕГЭ.

2.1. Преимущества программной генерации заданий

На примере предыдущей задачи было явно показано превосходство шаблонов над заданиями из Открытого Банка Заданий, а именно:

1. Большое количество разнообразных задач одного типа.
2. Простота и скорость написания шаблонов.
3. Невозможность нахождения учащимися ответов на задачи.

2.2. Вклад автора в расширение каталога

```
1 (function() { 'use strict'; retryWhileError(function() {
2   let s=sl(1, 200, 1);
3   let n=sl(1, 10, 1);
4   let a=sl(1, 15, 1);
5   let v=(a**2*n**2+2*s*a*n).sqrt();
6   let x=v/n;
7   genAssert(x>0, 'Скорость не может быть отрицательной');
8   genAssertZ1000(x, 'Скорость не должна быть слишком дробной');
9   let the_activeFloatingVehicle =
10     sklonlXkand(["лодка", "байдарка", "баржа", "яхта", "моторная лодка"].iz());
11 let the_humanSettlementDestination = sklonlXkand(["пункт", "город"].iz());
12 let the_orderToFind = decor.orderToFind.iz();
13   NATask.setTask({
14     text:
```

```

14     '' + the_activeFloatingVehicle.ie.toZagl() +' прошла против течения
реки ' + s +' км и вернулась в ' + the_humanSettlementDestination.ie +
15     ' отправления, затратив на обратный путь на ' + chislitlx(n, 'час') +'
меньше. ' + the_orderToFind.toZagl() +' скорость '+'
16     the_activeFloatingVehicle.re + ' в неподвижной воде, если скорость
течения равна ' + a + ' кмч/. Ответ дайте в кмч/.',
17     answers: x,
18 });
19 NATask.modifiers.allDecimalsToStandard(/*true*/);
20 }, 2000);})();

```

Листинг 1: 26586.js

Примеры генерируемых задач 26586.js

Моторная лодка прошла против течения реки 160 км и вернулась в пункт отправления, затратив на обратный путь на 1 час меньше. Вычислите скорость моторной лодки в неподвижной воде, если скорость течения равна 4 км/ч. Ответ дайте в км/ч. 36 26586.js

Байдарка прошла против течения реки 96 км и вернулась в пункт отправления, затратив на обратный путь на 4 часа меньше. Определите скорость байдарки в неподвижной воде, если скорость течения равна 1 км/ч. Ответ дайте в км/ч. 7 26586.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2     let n=s1(1, 3, 1);
3     let s=s1(10, 400, 1);
4     let a=s1(5,50,1);
5     let x=s1(1, 30, 1);
6     let k=n+s/(a-x)+s/(a+x);
7     genAssert(Number.isInteger(k),'Время не должно быть дробным');
8     genAssert(k>n,'Общее затраченное время больше стоянки');
9     genAssert(k<20,'Время стоянки не должно быть слишком большим');
10    let the_activeFloatingVehicle =

```

```

    sklonlxlkand(decor.activeFloatingVehicle.iz());
11     let the_humanSettlementDestination =
    sklonlxlkand(decor.humanSettlementDestination.iz());
12     let the_orderToFind = decor.orderToFind.iz();
13     NATask.setTask({
14         text:
15             '' + the_activeFloatingVehicle.ie.toZagl() + ' проходит по течению
реки до ' + the_humanSettlementDestination.re + ' назначения ' + s +
16             ' км и после стоянки возвращается в ' +
the_humanSettlementDestination.ie + ' отправления. ' +
17             the_orderToFind.toZagl() + ' скорость течения, если скорость ' +
the_activeFloatingVehicle.re + ' в неподвижной воде равна '+ a +
18             ' кмч/, стоянка длится ' + chislitlx(n, 'час') + ', ' + 'а в ' +
the_humanSettlementDestination.ie + ' отправления ' +
19             the_activeFloatingVehicle.ie + ' возвращается через ' + chislitlx(k,
'час') + '. '+
20             'Ответ дайте в кмч/.',
21             answers: x,
22             authors: ['VeronikaKit'],
23     });
24     NATask.modifiers.allDecimalsToStandard(/*true*/);
25 }, 2000);})();

```

Листинг 2: 26588.js

Примеры генерируемых задач 26588.js

Байдарка проходит по течению реки до деревни назначения 18 км и после стоянки возвращается в деревня отправления. Определите скорость течения, если скорость байдарки в неподвижной воде равна 24 км/ч, стоянка длится 1 час, а в деревня отправления байдарка возвращается через 3 часа. Ответ дайте в км/ч. 12 26588.js

Баржа проходит по течению реки до деревни назначения 160 км и после стоянки возвращается в деревня отправления. Определите скорость течения, если скорость

баржи в неподвижной воде равна 26 км/ч, стоянка длится 2 часа, а в деревня отправления баржа возвращается через 15 часов. Ответ дайте в км/ч. 6 26588.js

```
1 (function() { 'use strict'; retryWhileError(function() {
2   let s=slKrome(10, 300, 1);
3   let a=sl(1, 20);
4   let b=sl(1, 7);
5   let v=Math.sqrt(b**2*a**2+4*b*s*a);
6   let x=(-b*a+v)/(2*b);
7   genAssert(x>0,'Скорость не может быть отрицательной');
8   genAssertZ1000(x,'Скорость не может быть слишком дробной');
9   let the_berthForFloatingVehicle =
    sklonlXkand(decor.berthForFloatingVehicle.iz());
10  let the_waterbodyWithoutCurrent =
    sklonlXkand(decor.waterbodyWithoutCurrent.iz());
11  let the_activeFloatingVehicle =
    sklonlXkand(decor.activeFloatingVehicleF.iz());
12  let the_afterAWhile = decor.afterAWhile.iz();
13  let the_orderToFind = decor.orderToFind.iz();
14    NATask.setTask({
15      text:
16        '' + the_berthForFloatingVehicle.im.toZagl() + ' А и В расположены
на ' + the_waterbodyWithoutCurrent.pe + ', '+
17        'расстояние между ними равно ' + s + ' км. ' +
the_activeFloatingVehicle.ie.toZagl() + ' отправилась с постоянной скоростью
из А в В. ' +
18        the_afterAWhile.toZagl() + ' после прибытия она отправилась тем же путём
обратно со скоростью на ' + a + ' кмч/ больше прежней, '+
19        'сделав по пути остановку на ' + chislitlx(b, 'час') + '. В результате
она затратила на обратный путь столько же времени, '+
20        'сколько на путь из А в В. ' + the_orderToFind.toZagl() + ' скорость ' +
the_activeFloatingVehicle.re + ' на пути из А в В. Ответ дайте в кмч/.',
21      answers: x,
22    });
23  NATask.modifiers.allDecimalsToStandard(/*true*/);
```

```

24   NATask.modifiers.variativeABC();
25 }, 2000);})());

```

Листинг 3: 27482.js

Примеры генерируемых задач 27482.js

Пристани R и C расположены на озере, расстояние между ними равно 120 км. Баржа отправилась с постоянной скоростью из R в C . Через два дня после прибытия она отправилась тем же путём обратно со скоростью на 4 км/ч больше прежней, сделав по пути остановку на 1 час. В результате она затратила на обратный путь столько же времени, сколько на путь из R в C . Найдите скорость баржи на пути из R в C . Ответ дайте в км/ч. 20 27482.js

Пристани J и L расположены на водохранилище, расстояние между ними равно 168 км. Моторная лодка отправилась с постоянной скоростью из J в L . На следующий день после прибытия она отправилась тем же путём обратно со скоростью на 12 км/ч больше прежней, сделав по пути остановку на 7 часов. В результате она затратила на обратный путь столько же времени, сколько на путь из J в L . Найдите скорость моторной лодки на пути из J в L . Ответ дайте в км/ч. 12 27482.js

```

1  (function() { 'use strict'; retryWhileError(function() {
2    let s=sl(1, 200);
3    let n=sl(1, 10, 1);
4    let a=sl(1, 15, 1);
5    let v=(a**2*n**2+2*s*a*n).sqrt();
6    let x=v/n;
7    genAssert(x>0,'Скорость не может быть отрицательной');
8    genAssertZ1000(x,'Скорость не должна быть слишком дробной');
9    let the_activeFloatingVehicle =
10     sklonlxkand(decor.activeFloatingVehicleF.iz());
11 let the_humanSettlementDestination = sklonlxkand(["пункт","город"].iz());
12 let the_orderToFind = decor.orderToFind.iz();

```

```

12   NATask.setTask({
13     text:
14       the_activeFloatingVehicle.ie.toZagl() +' прошла против течения реки '
+ s +' км и вернулась в ' + the_humanSettlementDestination.ie +
15     ' отправления, затратив на обратный путь на ' + chislitlx(n, 'час') +'
меньше. ' + the_orderToFind.toZagl() +' скорость '+
16     the_activeFloatingVehicle.re + ' в неподвижной воде, если скорость
течения равна ' + a + ' кмч/. Ответ дайте в кмч/.',
17     answers: x,
18   });
19   NATask.modifiers.allDecimalsToStandard(/*true*/);
20 }, 2000);})();

```

Листинг 4: 111557.js

Примеры генерируемых задач 111557.js

Яхта прошла против течения реки 140 км и вернулась в пункт отправления, затратив на обратный путь на 8 часов меньше. Найдите скорость яхты в неподвижной воде, если скорость течения равна 1 км/ч. Ответ дайте в км/ч. 6 111557.js

Лодка прошла против течения реки 21 км и вернулась в пункт отправления, затратив на обратный путь на 9 часов меньше. Вычислите скорость лодки в неподвижной воде, если скорость течения равна 6 км/ч. Ответ дайте в км/ч. 8 111557.js

```

1  (function() { 'use strict'; retryWhileError(function() {
2    let s=sl(1, 200, 1);
3    let k=sl(1, 15, 1);
4    let b=sl(1, 24, 1);
5    let n=sl(1, 5, 1);
6    let x=slKrome(k, 5, 50);
7    let a=b-n-s/(x+k)-s/(x-k);
8    genAssert(Number.isInteger(a), 'Время не может быть дробным');
9    genAssert(a>0, 'Время отправления не может быть отрицательным');
10   genAssert(a<15, 'Время отправления не может быть слишком большим');
11   genAssert(b>a, 'Время отправления не может быть больше времени прибытия');

```

```

12     let the_activeFloatingVehicle =
        sklonlxlkand(["пароход", "теплоход", "каyak", "корабль", "паром", "катер"].iz());
13 let the_orderToFind = decor.orderToFind.iz();
14     NATask.setTask({
15         text:
16             '' + the_activeFloatingVehicle.ie.toZagl() + ' в ' + a + ':00 вышел
по течению реки из пункта А в пункт В, расположенный в ' +
17             s + ' км от А. Пробыв в пункте В ' + chislitlx(n, 'час') + ', ' +
the_activeFloatingVehicle.ie +
18             ' отправился назад и вернулся в пункт А в ' + b + ':00 того же дня. ' +
the_orderToFind.toZagl() + ' собственную скорость ' +
19             the_activeFloatingVehicle.re + 'в( кмч/), если известно, что скорость
течения реки ' + k + ' кмч/. Ответ дайте в кмч/.',
20         answers: x,
21     });
22     NATask.modifiers.allDecimalsToStandard(/*true*/);
23 }, 2000);})();

```

Листинг 5: 323375.js

Примеры генерируемых задач 323375.js

Корабль в 10 : 00 вышел по течению реки из пункта *A* в пункт *B*, расположенный в 45 км от *A*. Пробыв в пункте *B* 3 часа, корабль отправился назад и вернулся в пункт *A* в 22 : 00 того же дня. Определите собственную скорость корабля(в км/ч), если известно, что скорость течения реки 12 км/ч. Ответ дайте в км/ч. 18 323375.js

Теплоход в 1 : 00 вышел по течению реки из пункта *A* в пункт *B*, расположенный в 95 км от *A*. Пробыв в пункте *B* 2 часа, теплоход отправился назад и вернулся в пункт *A* в 15 : 00 того же дня. Найдите собственную скорость теплохода(в км/ч), если известно, что скорость течения реки 14 км/ч. Ответ дайте в км/ч. 24 323375.js

```

1 (function() { 'use strict'; retryWhileError(function() {

```

```

2  let s=sl(10, 300, 1);
3  let n=sl(1, 9);
4  let a=sl(1, 20);
5  let v=Math.sqrt(n**2*a**2+4*n*s*a);
6      let x=(-n*a+v)/(2*n);
7  genAssert(x>0,'Скорость не может быть отрицательной');
8  genAssertZ1000(x,'Скорость не может быть слишком дробной');
9      let the_berthForFloatingVehicle =
      sklonlхkand(decor.berthForFloatingVehicle.iz()); // пристаньпричал["",""]
10 let the_activeFloatingVehicle = sklonlхkand(
    ["пароход","теплоход","каяк","корабль","паром","катер"].iz());
11 let the_orderToFind = decor.orderToFind.iz(); //
    найдитеопределителевычислите["","",""]
12 let the_humanSettlementDestination =
    sklonlхkand(["пункт","город","село","деревню"].iz());
13     NATask.setTask({
14         text:
15             'От ' + the_berthForFloatingVehicle.im + ' А к ' +
the_berthForFloatingVehicle.im + ' В, расстояние между которыми равно '+s+'
км, '+
16             'отправился с постоянной скоростью первый ' +
the_activeFloatingVehicle.ie +', '+
17             'а через ' + chislitlx(n, 'час') + ' после этого следом за ним, '+
18             'со скоростью на ' + a + ' кмч/ больше, отправился второй. ' +
19             the_orderToFind.toZagl() + ' скорость первого ' +
the_activeFloatingVehicle.re +', если в ' +
20             the_humanSettlementDestination.ie + ' В оба '+
the_activeFloatingVehicle.re + ' прибыли одновременно. Ответ дайте в кмч/.',
21         answers: x,
22         authors: ['VeronikaKit'],
23     });
24     NATask.modifiers.allDecimalsToStandard(/*true*/);
25     NATask.modifiers.variativeABC();
26 }, 2000);})();
27 //VeronikaKit

```

Листинг 6: 26590.js

Примеры генерируемых задач 26590.js

От пристани U к пристани G , расстояние между которыми равно 224 км, отправился с постоянной скоростью первый катер, а через 8 часов после этого следом за ним, со скоростью на 9 км/ч больше, отправился второй. Определите скорость первого катера, если в город G оба катера прибыли одновременно. Ответ дайте в км/ч. 12 26590.js

От пристани X к пристани U , расстояние между которыми равно 120 км, отправился с постоянной скоростью первый корабль, а через 5 часов после этого следом за ним, со скоростью на 2 км/ч больше, отправился второй. Определите скорость первого корабля, если в пункт U оба корабля прибыли одновременно. Ответ дайте в км/ч. 6 26590.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2   let s=sl(20, 300, 1);
3   let n=sl(1, 5, 1);
4   let x=sl(1,15,1);
5   let v=slKrome(x, 3, 40);
6   let b=sl(1, 23, 1);
7     let a=b-n-s/(v+x)-s/(v-x);
8   genAssert(b>a,'Время отправления не должно быть больше времени прибытия');
9   genAssert(Number.isInteger(a),'Время не может быть дробным');
10  genAssert(a>0,'Время отправления не может быть отрицательным');
11  genAssert(a<15,'Время отправления не может быть слишком большим');
12  let the_activeFloatingVehicle = sklonlxkand(
13    ["лодка","байдарка","баржа","яхта","моторная лодка"].iz());
14  let the_humanSettlementDestination = sklonlxkand(["пункт","город"].iz());
15  let the_orderToFind = decor.orderToFind.iz();
16  NATask.setTask({
    text:

```

```

17     '' + the_activeFloatingVehicle.ie.toZagl() +' в ' + a + ':00 вышла из
    ' + the_humanSettlementDestination.re +
18     ' А в ' + the_humanSettlementDestination.ie +' В, расположенный в ' +
    s +
19     ' км от А. Пробыв в ' + the_humanSettlementDestination.pe +' В ' +
    chislitlx(n, 'час') + ', ' +
20     the_activeFloatingVehicle.ie +' отправилась назад и вернулась в ' +
    the_humanSettlementDestination.ie +
21     ' А в ' + b + ':00 того же дня. ' + the_orderToFind.toZagl() +
22     ' в( кмч/) скорость течения реки, если известно, '+
23     'что собственная скорость '+ the_activeFloatingVehicle.re + ' равна ' +
    v + ' кмч/.',
24     answers: x,
25     authors: ['VeronikaKit'],
26 });
27 NATask.modifiers.allDecimalsToStandard(/*true*/);
28 NATask.modifiers.variativeABC();
29 }, 20000);}());
30 //VeronikaKit
31 //Решу ЕГЭ 5997

```

Листинг 7: 5997.js

Примеры генерируемых задач 5997.js

Лодка в 7 : 00 вышла из города H в город G , расположенный в 54 км от H . Пробыв в городе G 4 часа, лодка отправилась назад и вернулась в город в 20 : 00 того же дня. Определите (в км/ч) скорость течения реки, если известно, что собственная скорость лодки равна 16 км/ч. 8 5997.js

Яхта в 1 : 00 вышла из города C в город B , расположенный в 45 км от C . Пробыв в городе B 3 часа, яхта отправилась назад и вернулась в город в 12 : 00 того же дня. Определите (в км/ч) скорость течения реки, если известно, что собственная скорость яхты равна 12 км/ч. 3 5997.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2     let a=sluchch(1,30);
3     let b=sluchch(1,30);
4     let h=sluchch(1,30);
5     let S=((a+b)/2)*h;
6     NATask.setTask({
7         text:'Площадь трапеции вычисляется по формуле  $S=\frac{a+b}{2}h$ , где
8          $a$  и  $b$  - основания трапеции,  $h$  - её высота. Пользуясь этой формулой,
9         найдите  $S$ , если
10         ['a='+a, 'b='+b, 'h='+h].shuffleJoin('$', '$')+ '$.',
11         answers: S,
12     });
13     NATask.modifiers.allDecimalsToStandard(/*true*/);
14 }, 20000);})();
15 //VeronikaKit
16 //РешуЕГЭ 506299

```

Листинг 8: 506299.js

Примеры генерируемых задач 506299.js

Площадь трапеции вычисляется по формуле $S = \frac{a+b}{2}h$, где a и b - основания трапеции, h - её высота. Пользуясь этой формулой, найдите S , если $b = 23$, $h = 29$, $a = 4$. 391,5 506299.js

Площадь трапеции вычисляется по формуле $S = \frac{a+b}{2}h$, где a и b - основания трапеции, h - её высота. Пользуясь этой формулой, найдите S , если $a = 4$, $b = 4$, $h = 29$. 116 506299.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2     let b=sl(2, 30);
3     let c=slKrome(b, 2, 30);
4     let e=slKrome([b,c], 2, 30);
5     let a=b*c*e;
6     let x=(b+1)*(c+1)*(e+1);
7     genAssert(b.isPrime(), 'Проверка на простое число');

```

```

8  genAssert(c.isPrime(), 'Проверка на простое число');
9  genAssert(e.isPrime(), 'Проверка на простое число');
10  NATask.setTask({
11    text:
12      'Если $p_1$, $p_2$ и $p_3$ - различные простые числа, то сумма всех
делителей числа $p_1 \\cdot p_2 \\cdot p_3$ равна $(p_1+1)(p_2+1)(p_3+1)$.
Найдите сумму всех делителей числа $'+a+'='+b+' \\cdot '+c+' \\cdot '+e+'$. ',
13    answers: x,
14    authors: ['VeronikaKit'],
15  });
16  NATask.modifiers.allDecimalsToStandard(/*true*/);
17 }, 2000);})();
18 //VeronikaKit
19 //Решу ЕГЭ база()507035

```

Листинг 9: 50703501.js

Примеры генерируемых задач 50703501.js

Если p_1 , p_2 и p_3 - различные простые числа, то сумма всех делителей числа $p_1 \cdot p_2 \cdot p_3$ равна $(p_1 + 1)(p_2 + 1)(p_3 + 1)$. Найдите сумму всех делителей числа $3335 = 29 \cdot 5 \cdot 23$. 4320 50703501.js

Если p_1 , p_2 и p_3 - различные простые числа, то сумма всех делителей числа $p_1 \cdot p_2 \cdot p_3$ равна $(p_1 + 1)(p_2 + 1)(p_3 + 1)$. Найдите сумму всех делителей числа $1173 = 23 \cdot 17 \cdot 3$. 1728 50703501.js

```

1  (function() { 'use strict'; retryWhileError(function() {
2    let a=sl(1, 30);
3    let b=slKrome(a, 1, 30);
4    let c=slKrome(b, 1, 30);
5    let e=[a,b,c].shuffle();
6    let x=2*(a*b+a*c+b*c);
7    NATask.setTask({
8      text:

```

```

9      'Площадь поверхности прямоугольного параллелепипеда с ребрами $a$, $b$ и
      $$$ вычисляется по формуле $$$ $S=2(ab+ac+bc)$ $. Найдите площадь поверхности
      прямоугольного параллелепипеда с ребрами $'+ e.joinLast('$, $' , '$ и $') +
      '$.',
10      answers: x,
11      authors: ['VeronikaKit'],
12  });
13  NATask.modifiers.allDecimalsToStandard(/*true*/);
14 }, 2000);})();
15 //VeronikaKit
16 //Решу ЕГЭ база() 511648

```

Листинг 10: 511648.js

Примеры генерируемых задач 511648.js

Площадь поверхности прямоугольного параллелепипеда с ребрами a , b и c вычисляется по формуле $S = 2(ab + ac + bc)$. Найдите площадь поверхности прямоугольного параллелепипеда с ребрами 9, 8 и 6. 348 511648.js

Площадь поверхности прямоугольного параллелепипеда с ребрами a , b и c вычисляется по формуле $S = 2(ab + ac + bc)$. Найдите площадь поверхности прямоугольного параллелепипеда с ребрами 3, 18 и 28. 1284 511648.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2   let a=sl(10, 150, 1);
3   let b=slKrome(a, 10, 150);
4   let c=slKrome([a,b], 10, 150);
5   let v=3*a*b*c/(a*b+b*c+a*c);
6   let d=['пути', 'трассы'].iz();
7   genAssert(v.isAlmostInteger(),'Скорость не может быть дробной');
8   NATask.setTask({
9     text:'Первую треть '+d+' автомобиль ехал со скоростью '+a+' кмч/, вторую
      треть – со скоростью '+b+' кмч/, а последнюю – со скоростью '+c+' кмч/.
      Найдите среднюю скорость автомобиля на протяжении всего пути. Ответ дайте в
      кмч/.' ,

```

```

10     answers: v,
11     authors: ['VeronikaKit'],
12 });
13 NATask.modifiers.allDecimalsToStandard(/*true*/);
14 }, 20000);}());
15 //РешуЕГЭ-99605

```

Листинг 11: 99605.js

Примеры генерируемых задач 99605.js

Первую треть трассы автомобиль ехал со скоростью 42 км/ч, вторую треть - со скоростью 56 км/ч, а последнюю - со скоростью 30 км/ч. Найдите среднюю скорость автомобиля на протяжении всего пути. Ответ дайте в км/ч. 40 99605.js

Первую треть трассы автомобиль ехал со скоростью 56 км/ч, вторую треть - со скоростью 63 км/ч, а последнюю - со скоростью 126 км/ч. Найдите среднюю скорость автомобиля на протяжении всего пути. Ответ дайте в км/ч. 72 99605.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2     let s=sl(20, 300, 1);
3     let n=slKrome(s, 250, 1000);
4     let x=2*s*n/(n+s);
5     let persona=sklonlxkand( ["путешественник","странник"].iz());
6     let voda=sklonlxkand(["море","океан"].iz());
7     let the_activeFloatingVehicle=sklonlxkand( ["яхта","катер"].iz());
8     genAssert(Number.isInteger(x),'Скорость не может быть дробной');
9     let the_orderToFind = decor.orderToFind.iz(); //
10     найдитеопределителывычислите["","",""]
11     NATask.setTask({
12         text:
13             persona.ie.toZagl()+ ' переплыл '+voda.ve+' на '+
14             the_activeFloatingVehicle.pe+' со средней скоростью '+s+
15             ' кмч/. Обратно он летел на спортивном самолете со скоростью '+ n+
16             ' кмч/. '+the_orderToFind.toZagl()+ ' среднюю скорость '+persona.re+' на
17             протяжении всего пути. Ответ дайте в кмч/.',

```

```

14     answers: x,
15     authors: ['VeronikaKit'],
16 });
17     NATask.modifiers.allDecimalsToStandard(/*true*/);
18 }, 20000);}());
19 //Решу ЕГЭ 99604

```

Листинг 12: 99604.js

Примеры генерируемых задач 99604.js

Путешественник переплыл море на катере со средней скоростью 228 км/ч. Обратно он летел на спортивном самолете со скоростью 494 км/ч. Найдите среднюю скорость путешественника на протяжении всего пути. Ответ дайте в км/ч. 312 99604.js

Странник переплыл море на яхте со средней скоростью 279 км/ч. Обратно он летел на спортивном самолете со скоростью 558 км/ч. Вычислите среднюю скорость странника на протяжении всего пути. Ответ дайте в км/ч. 372 99604.js

```

1 (function() { retryWhileError(function() {
2     let d=sluchch(1,20).pm();
3     let e=slKrome(d,1,20).pm();
4     let c=-d*e;
5     let b=-d-e;
6     genAssert(b**2-4*(-c)>0, 'Нужно два решения');
7     NATask.setEquationTask({
8         parts: ['x^2+'+b+'x', c],
9         roots: [d,e],
10    handleMultipleRoots: 'randomExceptList',
11    });
12 }, 20);}());
13 //Решу ОГЭ 137382
14 //VeronikaKit

```

Листинг 13: 137382.js

Примеры генерируемых задач 137382.js

Найдите корень уравнения $x^2 - 16x = -60$ Если корней несколько, в ответе укажите их произведение. 60 137382.js

Найдите корень уравнения $x^2 - x = 72$ Если корней несколько, в ответе укажите больший из них. 9 137382.js

```
1 (function() { retryWhileError(function() {
2     let d=sluchch(1,30).pm();
3     let e=slKrome(d,1,30).pm();
4     let c=d*e;
5     let b=d+e;
6     genAssert(b!=0, 'Множитель не должен равняться нулю');
7     NAtask.setEquationTask({
8         parts: ['x^2+' +c, b+'x'],
9         roots: [d,e],
10        handleMultipleRoots: 'randomExceptList',
11    });
12 }, 20);})();
13 //VeronikaKit
```

Листинг 14: 10001583.js

Примеры генерируемых задач 10001583.js

Найдите корень уравнения $x^2 - 54 = 15x$ Если корней несколько, в ответе укажите их произведение. -54 10001583.js

Найдите корень уравнения $19x = x^2 - 120$ Если корней несколько, в ответе укажите меньший из них. -5 10001583.js

```
1 (function() { retryWhileError(function() {
2     let a=sluchch(1,20).pm();
3     let b=sluchch(1,40).pm();
4     let d=0;
5     let e=-b/a;
```

```

6     genAssertZ1000(e, 'Корень не должен быть слишком дробным');
7     NAtask.setEquationTask({
8         parts: [a+'x^2+'+b+'x', 0],
9         roots: [d,e],
10        handleMultipleRoots: 'randomExceptList',
11    });
12 }, 20);})();
13 //VeronikaKit
14 //РешуОГЭ 311447

```

Листинг 15: 311447.js

Примеры генерируемых задач 311447.js

Найдите корень уравнения $10x^2 - 17x = 0$ Если корней несколько, в ответе укажите их произведение. 0 311447.js

Найдите корень уравнения $5x^2 - x = 0$ Если корней несколько, в ответе укажите их сумму. 0,2 311447.js

```

1 (function() {
2     retryWhileError(function() {
3         let a = sluchch(1, 10).pm();
4         let c = sluchch(1, 40).pm();
5         let b = (a + c).pm();
6         let v = b ** 2 - 4 * a * c;
7         let d = (-b + Math.sqrt(v)) / (2 * a);
8         let e = -(b + Math.sqrt(v)) / (2 * a);
9         genAssert(v >= 0, 'Подкоренное выражение не может быть отрицательным');
10        genAssert(b != 0, 'Нужно два решения');
11        genAssertZ1000(d, 'Корень не должен быть слишком дробным');
12        genAssertZ1000(e, 'Корень не должен быть слишком дробным');
13        NAtask.setEquationTask({
14            parts: [a + 'x^2+' + b + 'x+' + c, 0],
15            roots: [d, e],
16            handleMultipleRoots: 'randomExceptList',

```

```

17     });
18     }, 20);
19 })();
20 //VeronikaKit
21 //РешуОГЭ 311438

```

Листинг 16: 311438.js

Примеры генерируемых задач 311438.js

Найдите корень уравнения $-10x^2 - 9x + 19 = 0$ Если корней несколько, в ответе укажите их произведение -1,9 311438.js

Найдите корень уравнения $x^2 + 18x - 19 = 0$ Если корней несколько, в ответе укажите меньший из них. -19 311438.js

```

1  (function() { retryWhileError(function() {
2      let a=sluchch(1,20).pm();
3      let b=sluchch(1,20).pm();
4      let c=sluchch(1,50).pm();
5      let d=sluchch(1,50).pm();
6      let e=-b/a;
7      let f=-d/c;
8      genAssertZ1000(e, 'Корень не должен быть слишком дробным');
9      genAssertZ1000(f, 'Корень не должен быть слишком дробным');
10     NATask.setEquationTask({
11         parts: ['('+a+'x'+b+')('+c+'x'+d+')', '0'],
12         roots: [e,f],
13         handleMultipleRoots: 'randomExceptList',
14
15     });
16 }, 100);})();
17 //VeronikaKit
18 //369496

```

Листинг 17: 369496.js

Примеры генерируемых задач 369496.js

Найдите корень уравнения $(10x-1)(16x+16) = 0$ Если корней несколько, в ответе укажите их произведение. -0,1 369496.js

Найдите корень уравнения $(2x-6)(12x-39) = 0$ Если корней несколько, в ответе укажите их сумму. 6,25 369496.js

```
1 (function() { retryWhileError(function() {  
2     let d=sluchch(1,30).pm();  
3     let e=slKrome(d,1,30).pm();  
4     let c=d*e;  
5     let b=d+e;  
6     genAssert(c!=0, 'Нужно два решения');  
7     genAssert(b!=0, 'Нужно два решения');  
8     genAssertZ1000(d, 'Корень не должен быть слишком дробным');  
9     genAssertZ1000(e, 'Корень не должен быть слишком дробным');  
10    NATask.setEquationTask({  
11        parts: ['x^2+'+c, b+'x'],  
12        roots: [d,e],  
13        handleMultipleRoots: 'randomExceptList',  
14    });  
15 }, 20);})();  
16 //VeronikaKit  
17 //РешуОГЭ 311495
```

Листинг 18: 311495.js

Примеры генерируемых задач 311495.js

Найдите корень уравнения $x^2 + 315 = -36x$ Если корней несколько, в ответе укажите меньший из них. -21 311495.js

Найдите корень уравнения $-2x = x^2 - 440$ Если корней несколько, в ответе укажите их сумму. -2 311495.js

```
1 (function() { retryWhileError(function() {
```

```

2   let a=sluchch(1,30).pm();
3   let b=sluchch(1,30).pm();
4   let c=sluchch(1,30).pm();
5   let e=sluchch(1,30).pm();
6   let v=sluchch(1,30).pm();
7   let f=sluchch(1,30).pm();
8   let d=(v-a-e*b)/(b*c-f);
9   genAssertZ1000(d, 'Корень не должен быть слишком дробным');
10  NATask.setEquationTask({
11      parts: [a+'+'+b+'('+[c+'x',e].slag()+')', v+'+'+f+'x'],
12      roots: d,
13  });
14
15 }, 20000);})();
16 //137371

```

Листинг 19: 137371.js

Примеры генерируемых задач 137371.js

Найдите корень уравнения $14 + 14(-7-x) = -20 + 18x$ -2 137371.js

Найдите корень уравнения $-2-30x = -21-5(14x+17)$ -2,6 137371.js

```

1  (function() { retryWhileError(function() {
2      let a=sluchch(1,30).pm();
3      let b=sluchch(1,30).pm();
4      let c=sluchch(1,30).pm();
5      let e=sluchch(1,30).pm();
6      let v=sluchch(1,30).pm();
7      let f=sluchch(1,30).pm();
8      let g=sluchch(1,30).pm();
9      let d=(v-b-c*e+f*g)/(a+c+f);
10     genAssertZ1000(d, 'Корень не должен быть слишком дробным');
11     NATask.setEquationTask({
12         parts: [a+'x'+b+'+'+c+'(x'+e+')', f+'('+g+'-x)+'+v'],
13         roots: d,

```

```

14     });
15
16 }, 20);})();
17
18
19 //Решу ОГЭ 338495
20 //VeronikaKit

```

Листинг 20: 338495.js

Примеры генерируемых задач 338495.js

Найдите корень уравнения $-3x-4-14(x-25) = 13(15-x)-2$ 38,25 338495.js

Найдите корень уравнения $-15(5-x) + 13 = 24x-11 + 25(x+17)$ -14 338495.js

```

1 (function() { retryWhileError(function() {
2     let a=sluchch(1,30).pm();
3     let b=sluchch(1,30).pm();
4     let c=sluchch(1,30).pm();
5     let e=sluchch(1,30).pm();
6     let v=sluchch(1,30).pm();
7     let f=sluchch(1,30).pm();
8     let d=(v-b-c+f*e)/(a+1+e);
9     genAssertZ1000(d, 'Корень не должен быть слишком дробным');
10    NATask.setEquationTask({
11        parts: [a+'x'+b+'+(x'+c+')', e+'('+f+'-x)'+v],
12        roots: d,
13    });
14
15 }, 20);})();
16 //338480

```

Листинг 21: 338480.js

Примеры генерируемых задач 338480.js

Найдите корень уравнения $-13x-27+(x-15)=10(9-x)+24$ -78 338480.js

Найдите корень уравнения $-15x-29+(x-2)=-11(-17-x)+30$ -9,92 338480.js

```
1 (function() { retryWhileError(function() {
2     let a=sluchch(1,30).pm();
3     let b=slKrome(a,1,30).pm();
4     let d=(-Math.pow(a,2)-b**2)/(2*a+2*b);
5     genAssertZ1000(d, 'Корень не должен быть слишком дробным');
6     NATask.setEquationTask({
7         parts: ['(x+'+a+')^2+(x+'+b+')^2',2+'x^2'],
8         roots: d,
9     });
10
11 }, 20000);}());
12 //338494
```

Листинг 22: 338494.js

Примеры генерируемых задач 338494.js

Найдите корень уравнения $(x-15)^2+(x+20)^2=2x^2$ -62,5 338494.js

Найдите корень уравнения $(x-6)^2+(x-19)^2=2x^2$ 7,94 338494.js

```
1 (function() { retryWhileError(function() {
2     let a=sluchch(1,10).pm();
3     let b=sluchch(1,10).pm();
4     let c=sluchch(1,10).pm();
5     let e=(a**2+c);
6     let f=2*a*b;
7     let g=b**2;
8     let n=f**2-4*e*g;
9     let d=(-f+Math.sqrt(n))/(2*a**2+2*c);
10    let h=(-f-Math.sqrt(n))/(2*a**2+2*c);
11    genAssert(n>0, 'Дискриминант не может быть отрицательным');
```

```

12     genAssertZ1000(d, 'Корень не должен быть слишком дробным');
13     genAssertZ1000(h, 'Корень не должен быть слишком дробным');
14     NATask.setEquationTask({
15         parts: ['('+a+'x'+b+')^2'+c+'x^2',0],
16         roots: [d,h],
17     });
18
19 }, 2000);})();
20 //VeronikaKit

```

Листинг 23: 1111555.js

Примеры генерируемых задач 1111555.js

Найдите корень уравнения $(-10x-3)^2-4x^2=0$ Если корней несколько, в ответе укажите их сумму. -0,625 1111555.js

Найдите корень уравнения $(3x+2)^2-4x^2=0$ Если корней несколько, в ответе укажите их сумму. -2,4 1111555.js

```

1  (function() { retryWhileError(function() {
2      let a=sluchch(1,30).pm();
3      let b=slKrome(Math.abs(a),1,30).pm();
4      let c=[1,-1].iz();
5      let e=[1,-1].iz();
6      let d;
7      if (c!=e){
8          d=(a-b)/(c-e);
9      } else if (c!=-e){
10         d=-(a+b)/(c+e);
11     }
12     genAssertZ1000(d,'Корень не должен быть слишком дробным');
13     NATask.setEquationTask({
14         parts: [[e+'x',a].slag(), [c+'x',b].slag()],
15         wrapper: ['(', ')^2'],
16         roots: [d]

```

```

17     });
18 }, 1000);})();
19 //VeronikaKit
20 //РешуОГЭ 338526

```

Листинг 24: 338526.js

Примеры генерируемых задач 338526.js

Найдите корень уравнения $(-20-x)^2 = (x-22)^2$ 1 338526.js

Найдите корень уравнения $(-29+x)^2 = (-24-x)^2$ 2,5 338526.js

```

1 (function() { retryWhileError(function() {
2     let b=sluchch(1,30).pm();
3     let a=slKrome(b,1,30).pm();
4     let e=sluchch(1,30).pm();
5     let d=(b*b-e*a)/(a-b);
6     genAssertZ1000(d, 'Корень не должен быть слишком дробным');
7     NATask.setEquationTask({
8         parts: ['\\frac{'+a+'}{'+x+'+b+'}', '\\frac{'+b+'}{'+x+'+e+'}'],
9         roots: d,
10    });
11 }, 2000);})();
12 //VeronikaKit
13 //Решу ОГЭ 338482

```

Листинг 25: 338482.js

Примеры генерируемых задач 338482.js

Найдите корень уравнения $\frac{27}{x+13} = \frac{13}{x-8}$ 27,5 338482.js

Найдите корень уравнения $\frac{21}{x+5} = \frac{20}{x+21}$ -341 338482.js

```

1 (function() { retryWhileError(function() {
2     let c=sluchch(1,30).pm();
3     let a=sluchch(1,30).pm();

```

```

4   let b=sluchch(1,30).pm();
5   let e=sluchch(1,30).pm();
6   let g=slKrome(c,1,30).pm();
7   let f=slKrome(g,1,30).pm();
8   let d=-(b*g+e*c*g)/(a*g-c*f);
9   genAssertZ1000(d, 'Корень не должен быть слишком дробным');
10  NATask.setEquationTask({
11      parts:
12      [['\\frac{' + a + 'x' + b + '}' + c + '}', e].slag(), '\\frac{' + f + 'x' + '}' + g + '}''],
13      roots: d,
14  });
15 }, 2000);})();
16 //VeronikaKit
17 //1115560

```

Листинг 26: 1115560.js

Примеры генерируемых задач 1115560.js

Найдите корень уравнения $\frac{5x}{9} = 24 + \frac{5x-12}{-21}$ 30,96 1115560.js

Найдите корень уравнения $-24 + \frac{-10x+22}{-9} = \frac{30x}{-27}$ 11,9 1115560.js

```

1  (function() { retryWhileError(function() {
2      let c=sluchch(1,30).pm();
3      let a=slKrome(Math.abs(c),1,30).pm();
4      let b=sluchch(1,30).pm();
5      let e=slKrome(Math.abs(b),1,30).pm();
6      let d=(b*c-e*a)/(a-c);
7      genAssertZ1000(d, 'Корень не должен быть слишком дробным');
8      NATask.setEquationTask({
9          parts: ['\\frac{' + a + '}' + c + '}', '\\frac{' + c + '}' + e + '}''],
10         roots: d,
11     });
12
13 }, 2000);})();
14 //VeronikaKit

```

Листинг 27: 311381.js

Примеры генерируемых задач 311381.js

Найдите корень уравнения $\frac{11}{x+7} = \frac{13}{x+18}$ 53,5 311381.js

Найдите корень уравнения $\frac{9}{x-8} = \frac{18}{x-13}$ 3 311381.js

3. Рефакторинг программного кода в среде разработки на JavaScript

Рефакторинг кода представляет собой один из ключевых аспектов программной инженерии, который зачастую откладывается на более поздние сроки или полностью игнорируется в угоду оперативным задачам разработки. Распространённые утверждения, такие как <<у нас нет времени на рефакторинг>>, <<код функционирует корректно, зачем вносить изменения>> или <<рефакторинг является излишеством для идеалистов>>, свидетельствуют о непонимании его стратегической значимости. Подобное отношение к процессу рефакторинга неизбежно приводит к накоплению технического долга, который в конечном итоге проявляется в виде серьёзных проблем: замедления темпов разработки, увеличения количества программных ошибок и, в крайних случаях, необходимости полного переписывания программного продукта.

В рамках данной главы проводится детальный анализ причин, по которым рефакторинг в языке программирования JavaScript представляет собой не просто косметическое улучшение исходного кода, а критически важную инженерную практику, оказывающую прямое влияние на успешность проекта, производительность команды разработчиков и общее качество программного продукта.

Рефакторинг определяется как процесс систематической реструктуризации существующего программного кода без изменения его внешнего поведения. Ключевым аспектом данного определения является требование сохранения функциональной эквивалентности программы до и после преобразований. Рефакторинг не предусматривает добавления новой функциональности и не направлен на исправление программных ошибок. Основная цель рефакторинга заключается в улучшении внутренней архитектуры кода, повышении его читаемости, упрощении процесса сопровождения и обеспечении возможности последующего расширения.

Язык программирования JavaScript характеризуется динамической типизаци-

ей и отсутствием строгой системы типов, что предоставляет разработчику значительную степень свободы при реализации программных решений. Данная свобода одновременно является как преимуществом, так и потенциальным источником проблем. С одной стороны, использование JavaScript позволяет быстро создавать прототипы и реализовывать программные идеи. С другой стороны, существует высокая вероятность создания кода, который функционирует корректно, но обладает крайне низкими характеристиками в части поддерживаемости в долгосрочной перспективе.

Выделяют следующие факторы, обуславливающие критическую важность рефакторинга в среде разработки на языке JavaScript:

- **Отсутствие строгой типизации.** В языках со статической типизацией (например, Java, TypeScript) значительное количество ошибок выявляется на этапе компиляции. В языке JavaScript все проверки типов осуществляются во время выполнения программы, что существенно повышает риск возникновения ошибок, которые могут проявиться только при определённых условиях выполнения. Рефакторинг способствует компенсации отсутствия типизации путём создания самодокументируемого кода с семантически значимыми именами переменных и функций, а также применения защитного программирования с проверками типов и значений.
- **Асинхронная природа языка.** JavaScript изначально разрабатывался как язык для асинхронного программирования в среде веб-браузера. Эволюция подходов к обработке асинхронных операций прошла через несколько этапов: использование колбэк-функций (<<callback hell>>), применение объектов Promise и внедрение асинхронных функций (async/await). Отсутствие должной структуризации асинхронного кода приводит к формированию нечитаемых конструкций, которые практически невозможно поддерживать и отлаживать. Рефакторинг асинхронного кода позволяет преобразовать запутанные цепочки колбэков в читаемые и последовательные конструкции.

- **Динамическая эволюция языка.** JavaScript является активно развивающимся языком программирования. Стандарт ECMAScript обновляется ежегодно, внося новые возможности и улучшения. Код, написанный 5--10 лет назад, может использовать устаревшие паттерны и конструкции, для которых в современных версиях языка существуют более элегантные и эффективные решения. Рефакторинг обеспечивает модернизацию кодовой базы путём применения современных возможностей языка.
- **Значительный размер современных веб-приложений.** Современные веб-приложения часто содержат десятки или даже сотни тысяч строк исходного кода. Использование фреймворков, таких как React, Angular и Vue, позволяет создавать сложные одностраничные приложения (SPA), функциональность которых сопоставима с настольными программами. Отсутствие регулярного рефакторинга приводит к тому, что кодовая база становится неуправляемой: время на понимание кода увеличивается экспоненциально, внесение изменений требует всё больше усилий, количество регрессионных ошибок растёт, а новые разработчики не могут быстро адаптироваться к проекту.

4. Рефакторинг заданий ОГЭ и ЕГЭ

В процессе разработки возникает необходимость постоянного совершенствования качества и актуализации исходного кода. В контексте проекта генерации экзаменационных заданий рефакторинг приобретает особую значимость, поскольку от качества кода напрямую зависят корректность сгенерированных задач, возможность их масштабирования и адаптации под изменяющиеся требования экзаменационных спецификаций. В рамках настоящей главы представлен вклад автора в рефакторинг программного кода проекта <<Час ЕГЭ>>. Рассматриваются конкретные примеры преобразования исходного кода генераторов математических задач, демонстрирующие применение ключевых техник рефакторинга.

4.1. Вклад автора в рефакторинг на проекте

```
1 (function() { 'use strict'; retryWhileError(function() {
2     let a=sluchch(2,9).pm();
3     let g=sluchch(1,9).pm();
4     let d=sluchch(2,2*a*g-1).pm();
5     let b=(2*a*g+d);
6     let f=sluchch(1,a*g*g-1).pm();
7     let c=(a*g*g+f);
8     let h=2*(a*(c-f)).sqrt();
9     let m1=[d-h,d+h];
10    let t1=['меньше','больше'];
11    let v1=sluchch(1);
12    NATask.setTask({
13        text: ('Прямая  $y = ' + d + 'x' + f + '$ является касательной к графику функции$ 
 $y = ' + a + 'x^2 + bx' + c + '$. Найдите  $b$ , зная, что оно  $' + t1[v1] + '$ 
 $' + sluchch(m1[0]+1, m1[1]-1) + '$.$ '), plusminus(),
14        answers: m1[v1],
15    });
16    NATask.modifiers.allDecimalsToStandard(/*true*/);$ 
```

```
17 }, 20000);}());
```

Листинг 28: refactoring2.js

Примеры генерируемых задач refactoring2.js

Прямая $y = -x + 12$ является касательной к графику функции $y = 2x^2 + bx + 44$.
Найдите b , зная, что оно больше -14 . 15 refactoring2.js

Прямая $y = -56x + 95$ является касательной к графику функции $y = -5x^2 + bx - 340$. Найдите b , зная, что оно больше -58 . -126 refactoring2.js

```
1 (function() { 'use strict'; retryWhileError(function() {
2     let a=sluchch(2,9).pm();
3     let g=sluchch(1,9).pm();
4     let d=sluchch(2,2*a*g-1).pm();
5     let b=(2*a*g+d);
6     let f=sluchch(1,a*g*g-1).pm();
7     let c=(a*g*g+f);
8     let m=[a,c,d,f];
9     let n=['a','c','d','f'];
10    let h=sluchch(0,3);
11    NATask.setTask({
12        text: ('Прямая $y='+((h-2)?(d):('d'))+'x'+((h-3)?(f):('f'))+
13        '$ является касательной к графику функции $y='+
14        ((h)?(a):('a'))+'x^{2}'+b+'x'+((h-1)?(c):('c'))+'$. Найдите
15        $'+n[h]+'$.').plusminus(),
16
17        answers: m[h],
18        authors: ['VeronikaKit'],
19    });
20    NATask.modifiers.allDecimalsToStandard(/*true*/);
21 }, 20000);}());
22 /*119973 120717 121211 121215 120719 120721 120723 120725 120727 120729
    120731
    120733 120735 120737 120739 120741 120743 120745 120747 120749 120751
```

	120753									
23	120755	120757	120759	120761	120763	120765	120767	120769	120771	120773
	120775									
24	120777	120779	120781	120783	120785	120787	120789	120791	120793	120795
	120797									
25	120799	120801	120803	120805	120807	120809	120811	120813	120815	120817
	120819									
26	120821	120823	120825	120827	120829	120831	120833	120835	120837	120839
	120841									
27	120843	120845	120847	120849	120851	120853	120855	120857	120859	120861
	120863									
28	120865	120867	120869	120871	120873	120875	120877	120879	120881	120883
	120885									
29	120887	120889	120891	120893	120895	120897	120899	120901	120903	120905
	120907									
30	120909	120911	120913	120915	120917	120919	120921	120923	120925	120927
	120929									
31	120931	120933	120935	120937	120939	120941	120943	120945	120947	120949
	120951									
32	120953	120955	120957	120959	120961	120963	120965	120967	120969	120971
	120973									
33	120975	120977	120979	120981	120983	120985	120987	120989	120991	120993
	120995									
34	120997	120999	121001	121003	121005	121007	121009	121011	121013	121015
	121017									
35	121019	121021	121023	121025	121027	121029	121031	121033	121035	121037
	121039									
36	121041	121043	121045	121047	121049	121051	121053	121055	121057	121059
	121061									
37	121063	121065	121067	121069	121071	121073	121075	121077	121079	121081
	121083									
38	121085	121087	121089	121091	121093	121095	121097	121099	121101	121103
	121105									
39	121107	121109	121111	121113	121115	121117	121119	121121	121123	121125
	121127									

```

40 121129 121131 121133 121135 121137 121139 121141 121143 121145 121147
    121149
41 121151 121153 121155 121157 121159 121161 121163 121165 121167 121169
    121171
42 121173 121175 121177 121179 121181 121183 121185 121187 121189 121191
    121193
43 121195 121197 121199 121201 121203 121205 121207 121209 121213*/

```

Листинг 29: refactoring3.js

Примеры генерируемых задач refactoring3.js

Прямая $y = dx + 209$ является касательной к графику функции $y = 6x^2 + 25x + 593$. Найдите d . -71 refactoring3.js

Прямая $y = 7x + 23$ является касательной к графику функции $y = -7x^2 - 49x + c$. Найдите c . -89 refactoring3.js

```

1 (function() {
2
3   var b=sluchch(2,5); //Коеф.
4   var c=sluchch(1,20); //Уровень в БОЛЬШЕМ сосуде
5
6   var v1=s11(); //из 0 - меньшего в больший
7   var v2=s11();
8   var v3=s11(); //площадь 1 -
9
10  var m='форму правильной
    '+'тре', 'четырёх', 'пяти', 'шести', 'семи'].iz()+'угольной призмы, ';
11  var g1=['цилиндрический сосуд, ', 'сосуд, имеющий '+m];
12  var g2=['цилиндрическом сосуде, ', 'сосуде, имеющем '+m];
13  var h=sklonlxkand(om.fluids.iz());
14
15  window.vopr.ver=[''+(!v1?c*b*b:c)];
16  window.vopr.txt='В '+g2[v2]+' уровень '+h.re+' достигает '+ (v1?c*b*b:c)+'
    см. На какой высоте будет находиться уровень '+

```

```

17      h.re+', если перелить содержимое первого сосуда во второй '+g1[v2]+'
18      '+
19      (function(){
20          var str = '';
21          if (v3){
22              str = 'площадь основания';
23          }else if (v2){
24              str = 'сторона основания';
25          }else{
26              str = 'диаметр';
27          }
28          return str;
29      })()+
30      ' которого в '+chislitlx(v3?b*b:b,'раз')+' '+
31      (v1?'больше':'меньше')+
32      ' '+
33      (function(){
34          var str = '';
35          if (v3){
36              str = 'площади основания';
37          }else if (v2){
38              str = 'стороны основания';
39          }else{
40              str = 'диаметра';
41          }
42          return str;
43      })()+ ' первого? Ответ выразите в сантиметрах.';
44
45      window.vopr.kat['log']=0;
46      window.vopr.kat['prz']=0;
47      window.vopr.kat['drs']=0;
48      window.vopr.kat['tri']=0;
49      })();

```

Примеры генерируемых задач refactoring5.js

В цилиндрическом сосуде уровень раствора достигает 4 см. На какой высоте будет находиться уровень раствора, если перелить содержимое первого сосуда во второй цилиндрический сосуд, площадь основания которого в 25 раз меньше площади основания первого? Ответ выразите в сантиметрах. 100 refactoring5.js

В сосуде, имеющем форму правильной треугольной призмы, уровень ртути достигает 36 см. На какой высоте будет находиться уровень ртути, если перелить содержимое первого сосуда во второй сосуд, имеющий форму правильной треугольной призмы, сторона основания которого в 3 раза больше стороны основания первого? Ответ выразите в сантиметрах. 4 refactoring5.js

```

1 (function() {
2
3     for(var x=0.5;!(x.isZ()&& x>0);){
4         var a=sluchch(2,10);
5         var t=sluchch(4,10);
6         var b=sluchch(2,t-1);
7         var x=a*t/b-a;
8     }
9
10    var t1=sluchch(om.transportm.ie.length-1);
11    var t3=om.transportm.r2[t1];
12    var t4=om.transportm.re[t1];
13
14    var t5=['первым','вторым'];
15    var p5=[x+a,x];
16    var v5=s11();
17
18    window.vopr.txt='Два '+t3+' одновременно отправились в

```

```

'+(x*t)+'километровый- пробег. Первый ехал со скоростью, на '+'
19         a+' кмч/ большей, чем скорость второго, и прибыл к финишу
на '+chislitlx(b,'час')+
20         ' раньше второго. Найти скорость '+'
21         t4+', пришедшего к финишу '+t5[v5]+'\'. Ответ дайте в
кмч/.';
22
23
24     window.vopr.ver=[p5[v5]];
25
26     window.vopr.kat['log']=0;
27     window.vopr.kat['prz']=0;
28     window.vopr.kat['drs']=0;
29     window.vopr.kat['tri']=0;
30     })();
31
32     //Обзад 626583 26584
33     //Николай Авдеев

```

Листинг 31: refactoring6.js

Примеры генерируемых задач refactoring6.js

Два доисторических омнибуса одновременно отправились в 15-километровый пробег. Первый ехал со скоростью, на 2 км/ч большей, чем скорость второго, и прибыл к финишу на 2 часа раньше второго. Найти скорость доисторического омнибуса, пришедшего к финишу вторым. Ответ дайте в км/ч. 3 refactoring6.js

Два "Москвича" одновременно отправились в 48-километровый пробег. Первый ехал со скоростью, на 6 км/ч большей, чем скорость второго, и прибыл к финишу на 4 часа раньше второго. Найти скорость "Москвича", пришедшего к финишу первым. Ответ дайте в км/ч. 12 refactoring6.js

```

1 (function() {
2     var b = 0.5, x, a, s;

```

```

3   for (; !(b.isZ() && b > 0);) {
4       x = sluchch(5, 20);
5       a = sluchch(2, x - 1);
6       s = sluchch(10, 200);
7       b = s / (x - a) - s / (x + a);
8   }
9   chas2.task.setCountableTask([
10      vel: 'скорость течения',
11      zna: a,
12      rod: 1,
13      nah: 1,
14      nmn: 'кмч/',
15  ], {
16      vel: 'скорость лодки в неподвижной воде',
17      zna: x,
18      rod: 1,
19      nah: 1,
20      nmn: 'кмч/'
21  }, {
22      utv: 'лодка затратила на обратный путь на ' + chislitlx(b, 'час') + '
меньше',
23      vpr: 'насколько меньше времени затратила лодка на обратный путь',
24      zna: b,
25      nah: 1
26  }, {
27      vel: 'суммарное пройденное лодкой расстояние',
28      zna: s * 2,
29      rod: 2,
30      nah: 1,
31      nmn: 'км'
32  }, ], {
33      preambula: 'Моторная лодка прошла против течения реки и вернулась в пункт
отправления. ',
34  });
35 })();

```

Листинг 32: refactoring7.js

Примеры генерируемых задач refactoring7.js

Моторная лодка прошла против течения реки и вернулась в пункт отправления. Известно, что скорость течения равна 3 км/ч. Насколько меньше времени затратила лодка на обратный путь, если скорость лодки в неподвижной воде составляет 10 км/ч, а суммарное пройденное лодкой расстояние составляет 364 км?

12 refactoring7.js

Моторная лодка прошла против течения реки и вернулась в пункт отправления. Чему равно суммарное пройденное лодкой расстояние, выраженное в км, если скорость лодки в неподвижной воде составляет 12 км/ч, а лодка затратила на обратный путь на 25 часов меньше? Скорость течения составляет 8 км/ч.

250 refactoring7.js

```

1 (function() { 'use strict'; retryWhileError(function() {
2   let all_sports, rus_sports, answ, is_rus;
3   do {
4     all_sports = sluchch(100, 900, 2);
5     rus_sports = sluchch(10, all_sports - 50);
6     is_rus = ['', 'не '].iz();
7     answ = is_rus === '' ? (rus_sports - 1) / (all_sports - 1) :
      (all_sports - rus_sports) / (all_sports - 1);
8   } while (answ.ts().length > 4)
9   let name = om.imenaj.ie.iz();
10  let game = om.sportparn.pe.iz();
11  let country = om.strany.re.iz();
12  NATask.setTask({
13    text: 'Перед началом первого тура чемпионата по ' + game +
14    ' участниц разбивают на игровые пары случайным образом ' +
15    'с помощью жребия. Всего в чемпионате участвует ' + chislitlx(all_sports,
      'спортсменка') +

```

```

16     ', среди которых ' + chislitlx(rus_sports, 'участница') + ' из ' +
country + ', ' +
17     'в том числе ' + name + '. Найдите вероятность того, что в первом туре ' +
name +
18     ' будет играть с какой-либо спортсменкой ' + is_rus + 'из ' + country +
'.'.
19         answers: answ,
20     });
21     NATask.modifiers.allDecimalsToStandard(/*true*/);
22 }, 2000);})();

```

Листинг 33: refactoring8.js

Примеры генерируемых задач refactoring8.js

Перед началом первого тура чемпионата по вольной борьбе участников разбивают на игровые пары случайным образом с помощью жребия. Всего в чемпионате участвует 326 спортсменов, среди которых 79 участниц из Венесуэлы, в том числе Мария. Найдите вероятность того, что в первом туре Мария будет играть с какой-либо спортсменкой не из Венесуэлы. 0.76 refactoring8.js

Перед началом первого тура чемпионата по настольному теннису участников разбивают на игровые пары случайным образом с помощью жребия. Всего в чемпионате участвует 566 спортсменов, среди которых 114 участниц из Англии, в том числе Екатерина. Найдите вероятность того, что в первом туре Екатерина будет играть с какой-либо спортсменкой из Англии. 0.2 refactoring8.js

```

1  (function() {
2      NAinfo.requireApiVersion(0, 2);
3
4      var stations = NLsets.alphabets.russian.getRandomItems(2);
5      stations[0] += '.';
6      stations[1] += '.';
7
8      var name = sklonlxkand(window.imenaj.ie.iz());

```

```

9
10 do{
11     var distanceBetweenStations = sl(1.5,10,0.5);
12     var distanceToLeftStation = sl(0.100,2.000,0.001);
13     var answer = distanceBetweenStations * distanceToLeftStation /
14     (distanceBetweenStations - 2 * distanceToLeftStation)
15 }while(!(1000*answer).isZ()  answer <= 0  answer >= 20);
16
17 var questionsAndAnswers = [
18     [
19         'Каково сейчас расстояние в( км) между ' + name.te + ' и автобусом? ',
20         answer + distanceToLeftStation,
21     ],
22 ];
23
24 if ( (1000*answer / distanceToLeftStation).isZ() ){
25     questionsAndAnswers.push([
26         [
27             'Во сколько раз скорость автобуса больше скорости велосипеда?',
28             'Во сколько раз скорость велосипеда меньше скорости автобуса?',
29             ].iz(),
30         answer / distanceToLeftStation,
31     ])
32 }
33
34 questionsAndAnswers = questionsAndAnswers.iz();
35
36
37 NATask.setTask({
38     text: 'Между остановками ' + stations[0] + ' и ' + stations[1] + ' ' +
39     chislitlx(distanceBetweenStations, 'километр') + ' прямой трассы. ' +
40     name.ie + ' вышла на велосипеде из леса между станциями в ' +
41     chislitlx(distanceToLeftStation*1000, 'метр', 'р')+
42     ' от ' + stations[0] + ' и увидела, что к ' + stations[0] + ' в

```

```

направлении к ' + stations[1] +
43     ' с постоянной скоростью подъезжает автобус, на который ' + name.de + '
    нужно успеть. '+
44     'Она заметила, что если она сейчас поедет в сторону ' + stations[0] + ',
    она окажется там одновременно с автобусом. ' +
45     'Но и если она поедет в сторону ' + stations[1] + ', она также окажется
    там одновременно с автобусом, '+
46     'который успеет преодолеть весь участок от ' + stations[0] + ' до ' +
    stations[1] +
47     ', не останавливаясь на остановке ' + stations[0] + ' ' +
48     questionsAndAnswers[0] +
49     ' Считайте(, что автобус и велосипед движутся с постоянными скоростями и
    останавливаются мгновенно.) ',
50     answers: questionsAndAnswers[1].ts(),
51     // TODO: во сколько раз коростьс призрака меньше скорости поезда?
52 });
53 })();
54
55 //Николай Авдеев

```

Листинг 34: refactoring15.js

Примеры генерируемых задач refactoring15.js

Между остановками Ш. и П. 7 километров прямой трассы. Наталия выехала на велосипеде из леса между станциями в 700 метрах от Ш. и увидела, что к Ш. в направлении к П. с постоянной скоростью подъезжает автобус, на который Наталии нужно успеть. Она заметила, что если она сейчас поедет в сторону Ш., она окажется там одновременно с автобусом. Но и если она поедет в сторону П., она также окажется там одновременно с автобусом, который успеет преодолеть весь участок от Ш. до П., не останавливаясь на остановке Ш. Во сколько раз скорость велосипеда меньше скорости автобуса? (Считайте, что автобус и велосипед движутся с постоянными скоростями и останавливаются мгновенно.) 1.25

refactoring15.js

Между остановками Ш. и А. 4,5 километра прямой трассы. Анастасия выехала на велосипеде из леса между станциями в 225 метрах от Ш. и увидела, что к Ш. в направлении к А. с постоянной скоростью подъезжает автобус, на который Анастасии нужно успеть. Она заметила, что если она сейчас поедет в сторону Ш., она окажется там одновременно с автобусом. Но и если она поедет в сторону А., она также окажется там одновременно с автобусом, который успеет преодолеть весь участок от Ш. до А., не останавливаясь на остановке Ш. Каково сейчас расстояние (в км) между Анастасией и автобусом? (Считайте, что автобус и велосипед движутся с постоянными скоростями и останавливаются мгновенно.) 0.475

refactoring15.js

```
1 (function() { 'use strict'; retryWhileError(function() {
2   let s=sl(20, 400, 1);
3   let n=sl(1, 7, 1);
4   let k=sl(5, 20, 1);
5   let v=sl(5, 60, 1);
6     let x=sl(1, [10, v-1].minE(), 1);
7   let a=slKrome(x, 0, 23);
8     let b=a+n+k/60+s/(v+x)+s/(v-x);
9   genAssert(b.isAlmostInteger(),'Время не может быть дробным');
10  genAssert(b<23,'Время прибытия не может быть слишком большим');
11  let the_activeFloatingVehicle = sklonlxkand(
12    ["лодка","байдарка","баржа","яхта","моторная лодка"].iz());
13  let the_humanSettlementDestination = sklonlxkand(
14    ["пункт","город","село"].iz());
15  let the_orderToFind = decor.orderToFind.iz(); //
16    найдитеопределитевычислите["","",""]
17  NATask.setTask({
18    text:
19      '' + the_activeFloatingVehicle.ie.toZagl() + ' в ' + a + ':00 вышла из
20      ' + the_humanSettlementDestination.re +
21      ' А в ' + the_humanSettlementDestination.ie + ' В, расположенн'+['ый',
```

```

    'ая', 'ое', 'ые'][the_humanSettlementDestination.rod] + ' в ' + s +
18     ' км от А. Пробыв в ' + the_humanSettlementDestination.pe + ' В ' +
chislitlx(n, 'час') + ' ' + chislitlx(k, 'минута') + ', ' +
19     the_activeFloatingVehicle.ie + ' отправилась назад и вернулась в ' +
the_humanSettlementDestination.ie +
20     ' А в ' + b + ':00 того же дня. ' + the_orderToFind.toZagl() +
21     ' в( кмч/) скорость течения реки, если известно, что собственная скорость '
+ the_activeFloatingVehicle.re +
22     ' равна ' + v + ' кмч/.',
23     answers: x,
24     authors: ['VeronikaKit'],
25 });
26 NATask.modifiers.allDecimalsToStandard(/*true*/);
27 }, 20000);}());
28 //Решу ЕГЭ 5999

```

Листинг 35: refactoring5999.js

Примеры генерируемых задач refactoring5999.js

Яхта в 4 : 00 вышла из города А в город В, расположенный в 59 км от А. Пробыв в городе В 5 часов 7 минут, яхта отправилась назад и вернулась в город А в 16 : 00 того же дня. Определите (в км/ч) скорость течения реки, если известно, что собственная скорость яхты равна 21 км/ч. 9 refactoring5999.js

Байдарка в 13 : 00 вышла из города А в город В, расположенный в 54 км от А. Пробыв в городе В 3 часа 18 минут, байдарка отправилась назад и вернулась в город А в 22 : 00 того же дня. Определите (в км/ч) скорость течения реки, если известно, что собственная скорость байдарки равна 19 км/ч. 1 refactoring5999.js

```

1 (function() {
2     retryWhileError(function() {
3         NAinfo.requireApiVersion(0, 2);
4
5         let A = sl(30,900); //объем резервуара базовый()

```

```

6   let b = sl(1,[A/30,29].minE(),0.01);//объем разница()
7   let x = sl(b+1,100,0.01);//объем ответ()
8   let n = (A*b)/(x*(x+b));//время заполнения разница()
9   genAssertZ1000(n,'Время слишком дробное: '+n);
10  let liquid = sklonlxkand(['вода','жидкость'].iz());
11  let tub_naz = sklonlxkand(['труба','шланг'].iz());
12  let prop = ['пропускает','прокачивает'].iz();
13  let rez = sklonlxkand(['резервуар','бассейн','бак','цистерна','бойлер','ём-
кость'].iz());
14  let v1=sl1();
15  let
tub_num=['перв'+['ый','ая','ое','ые'][tub_naz.rod],'втор'+['ой','ая','ое','ые']
16  let v2=sl1();
17  let dol=[['медленнее','дольше'].iz(),'меньше'],['быстрее','больше']];
18  let esli=['','если известно, что ','','если'].iz();
19  let ono=['он'+['','а','о','и'][tub_naz.rod]];
20
21  NATask.setTask({
22    text: tub_num[v2].toZagl()+' '+tub_naz.ie+' '+prop+' на
'+chislitlx(b, 'литр','v')+' в минуту '+dol[v2][1]+' , чем
'+tub_num[1-v2]+' . '+
23    'Сколько литров '+liquid.re+' в минуту '+prop+' '+tub_num[1-v1]+'
'+tub_naz.ie+esli+rez.ve+' объёмом '+chislitlx(A, 'литр','r')+' '+
24    [ono+' '+['заполняет','опустошает'].iz()+' на '+chislitlx(n,
'минута','v')+' '+dol[1-v1][0]+' , чем '+tub_num[v1]+' '+tub_naz.ie+'?',
25    ' '+tub_num[v1]+' '+tub_naz.ie+'
'+['заполняет','опустошает'].iz()+' на '+chislitlx(n, 'минута','v')+'
'+dol[v1][0]+'?'].iz(),
26    answers: v1==1 ? x : x+b,
27    authors: ['Aisse-258']
28  });
29  NATask.modifiers.assertSaneDecimals();
30  NATask.modifiers.allDecimalsToStandard();
31  }, 2000000);
32  })();

```

```

33
34 /*РешуЕГЭ: 26597: 5881 5883 39863 39867
35    509657 642406 5885 5887 5889 5891
36    5893 5895 39801 39803 39805 39807
37    39809 39811 39813 39815 39817 39819
38    39821 39823 39825 39827 39829 39831
39    39833 39835 39837 39839 39841 39843
40    39845 39847 39849 39851 39853 39855
41    39857 39859 39861 39865 642477 642547
42    642617 642687*/

```

Листинг 36: refactoring26597.js

Примеры генерируемых задач refactoring26597.js

Первый шланг пропускает на 2,75 литра в минуту меньше, чем второй. Сколько литров воды в минуту пропускает первый шланг, если известно, что бассейн объёмом 567 литров второй шланг опустошает на 0,66 минуты быстрее? 47,25 refactoring26597.js

Вторая труба пропускает на 3,5 литра в минуту больше, чем первая. Сколько литров жидкости в минуту пропускает вторая труба, если бассейн объёмом 147 литров она заполняет на 4,8 минуты быстрее, чем первая труба? 12,25 refactoring26597.js

```

1 (function() {
2   'use strict';
3   NAinfo.requireApiVersion(0, 0);
4   var tourists = om.rusbukv.iz();
5
6   do {
7     var reis = sluchch(3, 7);
8     var vsego = reis * sluchch(3, 20);
9     var answers = reis / vsego;
10  } while (((answers * 10000) % 100 !== 0));
11

```

```

12  var poryadok = ['последним', 'предпоследним'];
13  for (let i = 0; i < Math.ceil(vsego / reis); i++) {
14      poryadok.push(om.porchisl[i + 1].t[0]);
15  }
16
17  NATask.setTask({
18      text: 'В группе туристов ' + vsego + ' человек. Их вертолётom в несколько
приёмов забрасывают ' +
19      'в труднодоступный район по ' + chislitlx(reis, 'человек') + ' за рейс.
' +
20      'Порядок, в котором вертолёт перевозит туристов, случаен. ' +
21      'Найдите вероятность того, что турист ' + tourists + '. полетит ' +
poryadok.iz() + ' рейсом вертолёта.',
22      answers,
23  });
24  })();

```

Листинг 37: refactoring320194.js

Примеры генерируемых задач refactoring320194.js

В группе туристов 80 человек. Их вертолётom в несколько приёмов забрасывают в труднодоступный район по 4 человека за рейс. Порядок, в котором вертолёт перевозит туристов, случаен. Найдите вероятность того, что турист Ф. полетит семнадцатым рейсом вертолёта. 0,05 refactoring320194.js

В группе туристов 20 человек. Их вертолётom в несколько приёмов забрасывают в труднодоступный район по 5 людей за рейс. Порядок, в котором вертолёт перевозит туристов, случаен. Найдите вероятность того, что турист Д. полетит вторым рейсом вертолёта. 0,25 refactoring320194.js

```

1  (function() {
2      retryWhileError(function() {
3          NAinfo.requireApiVersion(0, 2);
4          'use strict';

```

```

5
6  const people1 = ['мальчиков', 'девочек'].iz();
7  const kolvomest = sluchch(5, 201);
8  const kolvopeople1 = kolvomest - 2;
9  let people2 = people1 === 'мальчиков' ? 'девочки' : 'мальчика';
10 let mest = people1 === 'мальчиков' ? 'обе' : 'оба';
11
12 const nayti = ['будут', 'не будут'].iz();
13 const answers1 = 2 / (kolvomest - 1);
14 const answ1 = answers1 * 10000; // можно умножать и на 1000, тогда ответ
будет максимум до сотых
15
16 genAssert(answ1 % 10 === 0, 'Ответ кратен 10');
17 genAssert(answers1.mzhd(0, 1), 'Ответ вне интервала (0; 1)');
18
19 NAtask.setTask({
20   text: За` круглый стол на ${chislitlx(kolvomest, 'стул')} в случайном
порядке рассаживаются ${kolvopeople1} ${people1} и 2 ${people2}. Найдите
вероятность того, что ${mest} ${people2} ${nayti} сидеть рядом.` ,
21   answers: nayti === 'будут' ? answers1 : 1 - answers1,
22   });
23 }, 100);
24 })();

```

Листинг 38: refactoring325904.js

Примеры генерируемых задач refactoring325904.js

За круглый стол на 11 стульев в случайном порядке рассаживаются 9 мальчиков и 2 девочки. Найдите вероятность того, что обе девочки не будут сидеть рядом. 0,8 refactoring325904.js

За круглый стол на 5 стульев в случайном порядке рассаживаются 3 мальчиков и 2 девочки. Найдите вероятность того, что обе девочки не будут сидеть рядом. 0,5 refactoring325904.js

Заключение

В ходе выполнения дипломной работы было написано:

- Текстовые задачи ЕГЭ — 15 шаблонов принято.
- Уравнения ОГЭ — 17 шаблонов.

Так же был произведен рефакторинг 20 кодов.

Все добавленные в проект задания можно использовать для составления контрольных работ, проведения текущего контроля знаний учащихся, подготовки к ЕГЭ.

В будущем планируется добавить в проект большее количество заданий ЕГЭ и ОГЭ различных типов.

Список литературы

- [1] Китаева В. Д., Суматохина А. С. Программная генерация задач ЕГЭ по математике на языке JavaScript по темам «Текстовые задачи» и «Вектора» // 75-я Международная студенческая научно-техническая конференция, посвящённая 95-летию АИРХ-АТИРПИХ-АГТУ : материалы конференции (Астрахань, 14–19 апреля 2025 г.). – Астрахань : АГТУ, 2025. – С. 810–813.
- [2] Тренажёр «Час ЕГЭ» [Электронный ресурс]. – URL: <https://math.vsu.ru/chas-ege/sh/katalog.html> (дата обращения: 11.06.2026).
- [3] Федеральный институт педагогических измерений : открытый банк заданий ЕГЭ [Электронный ресурс]. – URL: <https://fipi.ru/ege/otkrytyy-bank-zadaniy-ege> .
- [4] Момот Е. А., Арахов Н. Д. Разработка и внедрение ПО для сбора статистики результатов подготовки к ЕГЭ по математике профильного уровня // Актуальные проблемы прикладной математики, информатики и механики. – 2021. – С. 1–2.
- [5] Открытый банк задач ЕГЭ по математике. Профильный уровень [Электронный ресурс]. – URL: <https://prof.mathege.ru/> .
- [6] Единый государственный экзамен [Электронный ресурс] // Википедия : свободная энциклопедия. – URL: https://ru.wikipedia.org/wiki/Единый_государственный_экзамен .
- [7] Портал «Решу ЕГЭ» [Электронный ресурс]. – URL: <https://ege.sdangia.ru/problem?id=27074> (дата обращения: 11.06.2026).
- [8] Фаулер М. Рефакторинг: улучшение существующего кода. – СПб. : Питер, 2003. – 432 с.

- [9] Мартин Р. Чистый код: создание, анализ и рефакторинг. – М. : ДМК Пресс, 2019. – 464 с.
- [10] Макконнелл С. Совершенный код. – М. : Русская редакция, 2005. – 896 с.
- [11] Оппенгейм Д., Вудс Д. Разработка веб-приложений с помощью JavaScript и Node.js. – СПб. : Питер, 2020. – 576 с.
- [12] Крокфорд Д. Как устроен и работает JavaScript. – СПб. : Питер, 2019. – 240 с.
- [13] Прототипы заданий № 1–14 ЕГЭ [Электронный ресурс] // Easy-Physic.ru. – 2015. – URL: <https://www.easy-physic.ru/wp-content/uploads/2015/11/prototipy-zadaniy-1-14-ege-.pdf> .
- [14] Голубев А. А., Спасская Т. А. Сборник заданий по математике : учебное пособие [Электронный ресурс]. – Тверь : Тверской государственный университет, 2015. – 160 с. – ISBN 978-5-7609-0976-3. – URL: <https://www.elibrary.ru/item.asp?id=23321749> .